

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Every successful business begins as an idea — and most failed businesses begin the same way. Studies of venture failure consistently identify the same root cause: entrepreneurs commit time and capital to ideas that were never properly validated. An aspiring founder in a city such as Jos, Nigeria, who decides to start a food processing business faces a cascade of questions before spending a single naira. Is the problem real and painful? Is there demonstrable demand in this specific location? Who are the competitors — formal and informal — already serving this market? What are the realistic risks? How much capital is required, what licences must be obtained, and in what sequence should the venture be launched?

Answering these questions traditionally requires market research consultants, industry reports, and personal networks — resources that are expensive, slow, and largely unavailable to the small-scale entrepreneurs who need them most. The consequence is that venture planning in emerging markets is frequently driven by intuition rather than evidence, and avoidable business failures follow.

At the same time, the raw material needed for evidence-based venture planning has never been more abundant. Population statistics, pricing data, competitor listings, regulatory guidance, financing options, and consumer discussions are published openly across the web. Recent advances in artificial intelligence — particularly large language models (LLMs) — have made it possible to reason over this material directly, without first spending minutes retrieving it.

This project leverages these advances to design and implement **Thrace — a Startup Intelligence Platform for Business Discovery and Venture Planning**. Thrace allows anyone to describe a business idea in plain language — for example, *"AI-powered marketplace for Nigerian farmers"* — and receive, in seconds, a structured **Venture Intelligence Report**: an executive verdict, the opportunity, target customers, market sizing, the competitive landscape, the business model, key risks, a validation plan, and recommended next steps, followed by a short list of real source links. The platform additionally provides two related capabilities: **Company X-Ray**, which applies the same intelligence layer to analysing an existing company, and **Discovery**, which proposes business opportunities worth validating.

1.1.1 Design position: fast intelligence first

A deliberate architectural decision shapes the whole system and is worth stating early, because it is the opposite of the intuitive approach.

The intuitive approach to this problem is a research pipeline: a sequence of specialised analytical passes over a business idea, each retrieving material from the live web, followed by a synthesis pass that assembles the findings. Thrace takes the opposite position:

> **Fast intelligence first.**

A modern LLM already holds substantial pretrained knowledge of markets, sectors, business models and competitive dynamics. Thrace reasons from that knowledge immediately and answers in one streaming call, then attaches a small number of real source links retrieved in the background. Source retrieval never blocks the response; if it fails, the report still streams. There is no web crawling, no agent orchestration on the request path, and no model-generated URLs.

The decisive implementation choice is that the analyst runs with **no tools at all**. It cannot search and cannot browse, so report generation is a function of the model alone and of output length alone. This makes latency predictable and keeps the failure surface small. The cost is that the model has no fresh evidence; that cost is managed honestly rather than hidden, through constraints on what the analyst may claim, and partially offset by real source links attached afterwards.

1.2 Statement of the Problem

Aspiring entrepreneurs — particularly in emerging markets — face the following challenges when attempting to plan a new venture:

1. **No affordable validation:** Professional feasibility studies and market research are priced beyond the reach of small-scale founders, so ideas are launched on intuition alone.

1. **Data fragmentation:** The evidence needed to judge an idea — demand signals, local market statistics, competitor information, regulatory requirements, and cost benchmarks — is scattered across dozens of websites with no integration between them.

1. **Absence of structured judgement:** Even where information can be found, there is no accessible tool that converts it into a disciplined verdict — pursue, pivot, or drop — delivered in a consistent, directly comparable structure.

1. **Static, one-dimensional tools:** Existing business-planning tools produce generic template documents. They do not investigate, they do not consider the specific location of the venture, and they do not produce a structured analytical view of the opportunity.

1. **Untimely intelligence for existing businesses:** Even founders and investors researching an operating company must manually assemble competitor, sentiment, and performance information from fragmented sources.

1. **Opportunity discovery is manual:** A founder who does not yet have a specific idea has no way to find one. The result is that many viable opportunities are never considered at all.

There is, therefore, a clear need for a system that validates business ideas, delivers planning-stage intelligence at each step, applies the same analysis to researching existing

companies, and can propose opportunities worth investigating.

1.3 Aim and Objectives of the Study

Aim

The aim of this project is to design and implement a platform that validates business ideas and generates decision-ready venture planning intelligence quickly enough to be used conversationally — returning a structured report in seconds rather than minutes, with a small number of real source links, and without fabricating evidence.

Specific Objectives

The specific objectives of this project are to:

1. Design a single fast intelligence architecture serving three capabilities:
a primary **Venture Intelligence** report for new business ideas, a secondary **Company X-Ray** report for existing companies, and a **Discovery** capability proposing opportunities worth validating.
1. Develop the Venture Intelligence report to cover, in one pass:
a. an executive verdict (pursue / pivot / drop); b. the opportunity and the problem it addresses;
c. target customers; d. market sizing with assumptions stated; e. the competitive landscape and the gap a new entrant could exploit; f. the business model and the metric that decides whether it works; g. key risks with mitigations; h. a validation plan ordered cheapest-first, with falsification criteria; and i. recommended next steps.
1. Implement lightweight source collection that attaches a small number of genuine, clickable, deduplicated links to every report, never blocks generation, and cannot produce a fabricated URL.
1. Generate structured, human-readable reports streamed progressively so the user sees content immediately rather than waiting for completion.
1. Build an interactive, modern user interface — featuring a mode toggle between Venture Intelligence and Company X-Ray, a collapsible navigation sidebar with searchable, pinnable chat history, a report library, a discovery view, account-based sync, and light/dark theming.
1. Measure the system's latency by phase and optimise the measured bottleneck rather than an assumed one.

1.4 Significance of the Study

This project is significant in several ways:

- **For aspiring entrepreneurs:** The platform substitutes an expensive,

weeks-long feasibility study with an on-demand, structured report delivered in seconds — a fast enough turnaround that a founder can interrogate several ideas in one sitting, exposes the risks, and lays out a concrete validation plan before any capital is committed.

- **For small-business ecosystems in emerging markets:** By localising its intelligence (for example, Nigerian market context, naira cost estimates, CAC registration requirements), Thrace addresses a market segment that global research tools ignore.
- **For investors and analysts:** The Company X-Ray capability provides consolidated competitor, sentiment, and performance intelligence on existing companies, with genuine source links.
- **For founders without an idea:** The Discovery capability proposes business opportunities worth validating for a stated region or sector, closing a gap that idea-validation tools leave open.
- **For researchers:** The project contributes a working case study of a latency-optimised LLM product architecture, and a documented account of how measurement — rather than intuition — identified the true bottleneck in an LLM-based system.
- **For industry:** The system demonstrates how decision-support can be packaged as a consumer-grade product with a conversational experience.

1.5 Scope of the Study

The scope of this study covers the design and implementation of the Thrace platform as a full-stack application. **Venture Intelligence** accepts a free-text business idea and returns a structured report covering an executive verdict, the opportunity, target customers, market sizing, the competitive landscape, the business model, key risks, a validation plan, and recommended next steps, capped for latency. **Company X-Ray** analyses an existing company across competitor positioning, sentiment, and performance metrics. **Discovery** proposes business opportunities framed explicitly as hypotheses worth validating rather than findings. Every report ends with three to five real, deduplicated, clickable source links attached by the backend.

The system is implemented with a React 19 + TypeScript + Tailwind CSS v4 + shadcn/ui frontend; a Python FastAPI backend exposing Server-Sent Events (SSE) streaming; the Agno agent framework; Google Gemini language models (gemini-3.8-flash by default, with a latency-aware fallback pool; OpenAI-compatible providers also supported); and optional Firecrawl search for live source links. Chats persist both in the browser and server-side, with user accounts, Google sign-in, and password recovery.

The study does not cover continuous market monitoring, automatic trend detection, automated validation-experiment execution, or claim-level source provenance. These are identified as future work in Chapter Six.

1.6 Organisation of the Report

This report is organised into six chapters. **Chapter One** provides the introduction, background, problem statement, aim and objectives, significance, and scope. **Chapter Two** reviews literature on entrepreneurship and venture validation, business intelligence, web data acquisition, large language models, and multi-agent systems. **Chapter Three** presents the methodology, including the fast intelligence architecture, the shared analyst rules, the source-collection design, the model-selection strategy, and the user interface design. **Chapter Four** describes the implementation and presents the results, including measured latency, sample report structures and testing outcomes. **Chapter Five** discusses the findings, compares the platform with related systems, and reflects on strengths, limitations, and challenges. **Chapter Six** concludes the report and offers recommendations for future work.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews the literature and background concepts relevant to the design of a startup intelligence platform for business discovery and venture planning. The review is organised into six areas: entrepreneurship and venture validation; business intelligence and market analysis; web data acquisition and processing; large language models and generative AI; quantitative risk scoring; and multi-agent systems. The chapter concludes with a review of related systems and a statement of the research gap this project addresses.

2.2 Conceptual Review

2.2.1 Entrepreneurship and Venture Validation

Modern entrepreneurship theory places idea validation at the centre of new venture creation. The lean startup methodology argues that startups should treat ideas as hypotheses to be tested rapidly and cheaply, before committing significant resources. Customer development theory similarly insists that founders must first verify that a real, painful problem exists and that customers will pay for a solution, before building the product.

Validation, however, is only the first stage of venture planning. A validated idea must still be examined against its market (size, customer segments, pricing tolerance), its competition (direct, indirect, and informal alternatives), its risk profile, and its practical execution requirements — capital, funding options, go-to-market strategy, regulatory obligations, and a phased roadmap. In emerging markets this planning is complicated by thin formal data coverage: statistics about secondary cities, informal-sector competitors, and local purchasing power are scattered, inconsistent, or simply unpublished. Entrepreneurs in these environments routinely plan with little more evidence than personal observation, and venture failure rates reflect it.

A recurring theme in the literature is that the discipline of validation — explicit verdicts, quantified assessments, and staged planning — separates deliberate founders from gamblers. Yet the analytical labour required to do this discipline well is exactly what small founders cannot afford. This is the tension this project addresses: delivering consultant-grade validation and planning analysis at negligible marginal cost through automation.

2.2.2 Business Intelligence and Market Analysis

Business intelligence practice establishes a general principle that this project applies: an analysis is only as useful as the structure it imposes on incomplete information. Analysts working with sparse or partial data rely on a fixed analytical frame — defined questions, consistent categorisation, and explicit separation of observation from inference — because a free-form narrative invites the reader to accept conclusions that the evidence does not support.

Two properties of BI practice transfer directly to this problem. The first is **structural consistency**: the same analytical questions are answered in the same order every time, so that two analyses of different subjects can be compared. The second is **explicit provenance of uncertainty**: a professional analysis distinguishes what is known from what is estimated, and attaches an assumption to every figure it cannot source.

Thrace adopts both. Each report follows a fixed section list, and the analyst is required to label estimated figures and state the assumption behind them. Section 3.4 of Chapter Three describes how these constraints are implemented and why the language discipline is a functional requirement rather than a stylistic preference.

2.2.3 Web Data Acquisition and Processing

Three techniques are relevant to acquiring venture-planning evidence:

- **Web search** returns ranked results with titles, URLs and snippets, quickly and cheaply, without downloading page bodies.
- **Web crawling** systematically downloads pages by traversing links, providing the full text of a site rather than a snippet.
- **Web scraping/extraction** converts page content into clean text or structured fields by stripping markup and boilerplate.

The trade-off between them is central to this project's design. Search is fast and cheap but returns snippets; crawling yields full text but is slow, expensive, and open-ended, since a single site can expand into hundreds of pages. For a system whose output must appear in seconds, the cost profile favours search decisively, and the depth that crawling would provide is not the constraint that matters most.

In this project, the Firecrawl API is used for **search only**, and only for source collection. Search results supply genuine URLs which the backend attaches to the completed report; pages are not crawled and no analyst browses them. The analytical content of a report is produced by the model from its own pretrained knowledge, so no claim in the report depends on a retrieved page being read. This separation is deliberate: it is what allows source retrieval to be optional, and to fail without affecting the report. See Section 3.5.

2.2.4 Large Language Models and Generative AI

Large language models are deep neural networks trained on massive text corpora. Their relevance to this problem rests on three capabilities.

Pretrained domain knowledge. Language models encode substantial information about industries, business models, market structures and competitive dynamics acquired during pretraining. For well-trodden domains this is a usable analytical substrate without retrieval.

Instruction following. Modern instruction-tuned models accept a detailed specification and conform to it, which makes it possible to constrain an analyst's structure, register and honesty in a system prompt. This is the mechanism by which the report format and the language discipline in Section 3.4 are enforced.

Streaming generation. Tokens are produced incrementally, so a response can be transmitted as it is written. This converts generation latency from a blocking wait into a progressive experience and is the basis of the streaming protocol in Section 3.6.

Two limitations are equally relevant. Language models are prone to **hallucination** — producing plausible but unverified entities, figures and citations — which is the central risk in a business-advice context and the subject of Section 3.5. They are also **non-deterministic**: two runs on the same input may differ, which is why this project does not present a single composite score and instead decomposes judgement across named sections where a reader can locate and assess each dimension separately.

2.2.5 Quantitative Venture Assessment

Quantitative venture assessment has a long history, from credit-scoring techniques to weighted multi-criteria decision analysis. The consistent lesson is that decomposing a judgement into explicit, weighted dimensions improves calibration and transparency compared with holistic intuition. Classic frameworks — SWOT analysis, weighted scoring models, and stage-gate feasibility reviews — all share this decomposition principle.

This project retains the decomposition principle but not the composite score. A single weighted probability is avoided for a specific reason: the model supplies both the inputs and the arithmetic, and the inputs are estimates. A number with banded thresholds presents an appearance of precision that the underlying evidence does not support, and a reader has no way to audit the inputs. The report instead separates Opportunity, Target Customers, Market, Competition, Business Model, Key Risks and Validation Plan into distinct sections, so that a reader can see which dimensions are well evidenced and which are judgement, and can interrogate any of them with a follow-up question.

2.2.6 Multi-Agent Systems

A multi-agent system (MAS) is composed of multiple interacting intelligent agents, each with a scoped role and limited tools, coordinated to decompose a task that exceeds the competence of a single agent. The literature reports benefits — parallelism, specialisation, and fault isolation — alongside significant costs: coordination overhead, inter-agent error propagation, and markedly higher latency, since a sequence of agents multiplies the wall-clock time of the slowest one.

The design decision in Section 3.4 is made against this background. A staged agent architecture is appropriate when a task genuinely decomposes and when per-stage evidence retrieval is required. It is a poor fit when a single reasoning pass over pretrained knowledge can produce the deliverable, because the coordination cost is then paid without corresponding benefit.

Thrace uses one analyst with a shared system prompt across all three features; the three capabilities differ in their user prompt and output ceiling, not in execution strategy.

2.3 Review of Related Systems

2.3.1 Market-Research Databases

Commercial databases (CB Insights, Crunchbase) aggregate structured company records. They are strong on funded, developed-market companies and weak in exactly the segments this project targets: pre-seed ideas, informal local competition, and secondary cities in emerging markets, which are systematically under-recorded. They also answer questions about companies that already exist, not about whether a proposed idea should be pursued.

2.3.2 Business-Planning Software

Business Model Canvas tools and template planners structure the founder's own assumptions. They generate no external evidence; the output is only as good as the founder's prior knowledge, and the analytical work is not performed at all.

2.3.3 Social-Listening Tools

Brandwatch and comparable platforms monitor brand mentions across social channels. They serve established brands tracking their own perception and cover a single analytical dimension. The actor and the question are both different from an aspiring founder validating a new idea.

2.3.4 General-Purpose Conversational Assistants

The most important related system is not a specialised tool but the general conversational assistant, because it is what a founder will otherwise open. These systems can produce a competent business analysis, and on raw output quality they are difficult to beat. Their differences from a purpose-built platform are structural rather than qualitative:

- **No fixed analytical frame.** A conversational assistant answers the question asked. It does not reliably apply a consistent section structure, so two runs on the same idea may not be comparable.
- **No evidence boundary.** There is no separation between what the model knows and what it has been verified against, and no mechanism preventing invented citations.
- **Interaction cost.** Obtaining a structured report requires a sequence of prompt engineering by the user, who must specify format, depth, sections and constraints.
- **No continuity.** Analyses are not retained, versioned, or comparable over time, and cannot be watched for change.

2.3.5 Multi-Agent Intelligence Demonstrators

Open-source multi-agent demonstrators have shown that teams of tool-equipped LLM agents can produce competitor, sentiment, and metrics reports for a given company. These validate the

underlying architecture and directly informed the design of the Company X-Ray capability. The comparison is developed in Chapter Five.

2.4 Summary of the Gap

The review identifies the following gaps:

1. No existing consumer-accessible tool **validates a raw business idea** and then carries the analysis through market sizing, competition, business model, risk, and validation planning in one integrated, consistently structured flow.
1. Existing tools are **not location-aware** in emerging-market contexts; none reason about a specific city's market, costs, informal competitors, or regulatory steps (e.g., CAC registration in Nigeria).
1. Tools that analyse companies are **fragmented by dimension** (data, sentiment, or metrics) and manual in operation.
1. Where generative systems do offer venture analysis, they offer no **source-integrity guarantee** — the model may produce plausible but unverified URLs — and no fixed report structure, so that two analyses of the same idea are not comparable.
1. Retrieval-heavy architectures are **too slow for conversational use**, returning minutes after submission, which forecloses the rapid compare-several-ideas workflow that most founders actually want.
1. Few tools **propose opportunities** to a founder who does not yet have a specific idea.

This project addresses these gaps with a fast single-analyst platform that produces a fixed, decision-ready report within seconds, separates model knowledge from verified sources, guarantees real clickable links, and extends to opportunity discovery.

2.5 Summary

This chapter reviewed entrepreneurship and validation theory, business intelligence, web data acquisition, large language models, risk-scoring frameworks, and multi-agent systems, and examined related tools. The review established the gap: no fast, location-aware, source-integrity-guaranteed system validates business ideas and generates complete venture planning intelligence. The next chapter presents the methodology adopted to fill this gap.

METHODOLOGY

This chapter presents the methodology adopted for the design and implementation of the Thrace Startup Intelligence Platform for Business Discovery and Venture Planning. It describes the fast intelligence architecture, the shared analyst rules that govern output honesty, the source-collection design and its source-integrity guarantee, the model-selection strategy, the three feature prompts, the report generation format, the system interface, the technology stack, and the testing approach used to validate the implementation.

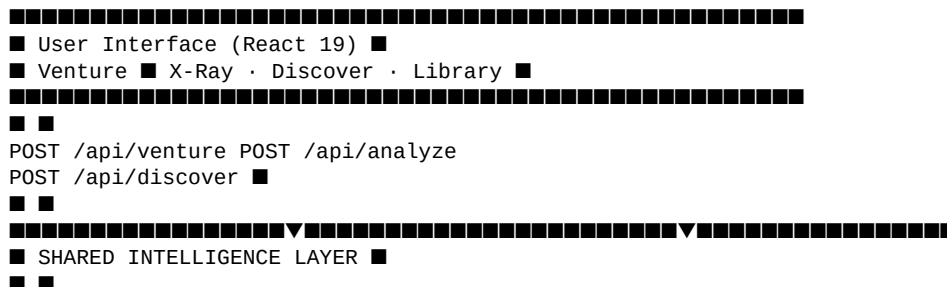
The project adopted a design-and-implementation methodology suited to producing a working software artefact. Development proceeded iteratively in five phases:

1. **System design** — defining the single-analyst architecture, the analyst honesty rules, the source-collection tiers, the API contract, and the user interface.

1. **Testing and validation** — verifying launch, streaming, source integrity, error handling, and end-to-end workflow execution for all three features.

3.3 System Architecture

The platform is built around a single shared intelligence layer serving three features. Each request is one streaming model call with no tools, followed by optional background source collection.



[illegible]

Figure 3.1: High-level architecture. One intelligence layer, three features, no tools on the request path.

3.4 Design of the Shared Intelligence Layer

Three features share one execution strategy. The design rests on a small number of decisions, each of which responds to an identified problem.

3.4.1 One streaming call, no tools

The analyst runs with **no tools at all**. It cannot search, cannot browse, and therefore cannot be delayed by, or fail because of, a retrieval backend. This is the single most important decision in the system: it makes report generation a function of the model alone, and makes generation time a function of output length alone.

The cost is that the model has no fresh evidence. That cost is managed honestly rather than hidden — see 3.4.2 and section 3.5.

3.4.2 Analyst honesty rules

One system prompt governs every feature, so the three differ only in their user prompt, never in their honesty constraints. It forbids fabrication of sources, URLs, citations, statistics,

companies, people and funding rounds; requires that estimates be labelled as estimates with their assumption stated; requires hedged language for judgements rather than asserted fact; forbids implying that research was performed; and requires direct, non-repetitive answer.

The rule that the analyst **may never write a URL** is load-bearing. It is what makes it safe to attach real sources: the model is not trusted with links because it is not permitted to produce any. Real links are supplied by the backend.

The prompt also states that another process attaches the links, which removes any incentive for the model to produce one.

3.4.3 Output ceilings

Generation time scales with output length, so each feature has a hard token ceiling: 1300 for Venture Intelligence, 1000 for Company X-Ray, 900 for Discovery, 500 for report Q&A. Dense markdown tokenises at roughly 4–6 characters per token on the model in use, so the venture ceiling corresponds to roughly a 5.8k-character report.

The ceilings are tuned to **complete** rather than to truncate. A report cut off mid-sentence reads as a broken product rather than a concise one. A truncation detector checks finished reports and logs the condition rather than shipping it silently, and the prompt states an explicit word budget so the model aims for concision rather than being merely clipped.

The detector is deliberately biased towards silence. Markdown legitimately ends without sentence punctuation — on a heading, a table row, a blockquote, a fence, or a short bolded label — and only a long plain-prose line ending on a bare word is treated as truncation.

3.4.4 Progressive output for Discovery

Discovery emits each opportunity as its block completes rather than after the whole response, so cards appear while the analyst is still writing. This requires no change to the streaming contract: the analyser tracks completed blocks in the accumulated text and yields a structured `idea` event for each.

3.5 Design of the Source-Collection Layer

Source collection is a separate concern from generation, and the design goal was to obtain genuine links **without ever delaying the report**.

3.5.1 The source-integrity guarantee

Three rules together guarantee that no fabricated URL can reach a user:

1. The model is forbidden by system prompt from writing a URL.
2. The renderer is the only component that emits a link, and it formats a list of structured source objects — it never accepts free text from the model.
1. Every URL is either returned by an actual search result, or is a known official organisation's root domain held in a curated list in source code.

3.5.2 Three tiers

Sources are assembled in descending relevance, each tier filling only what the tier above left short:

Tier	Source	Role
1	Live search results	Most specific; always first
2	Curated official bodies, scored against the query	Sector and regulator relevance
3	Broad cross-sector official bodies	Reaches the floor of three

The curated tier holds 21 official bodies spanning multilateral development finance, agriculture, industry, trade, health, digital connectivity, national regulators and development agencies. The result is bounded to a maximum of five and a minimum of three, deduplicated by URL, and the renderer re-applies the same rules so a caller that bypassed the collector still cannot emit a stub or an over-long list. An empty list renders nothing rather than a fabricated block.

A tier-2 body scores higher for a sector term (**fintech**, **agricultur**, **health**) than for a catch-all (**market**, **business**), and a local regulator receives a boost when the query names its country. A query of *"fintech in Lagos"* therefore leads with the Central Bank of Nigeria and the securities regulator, while *"agriculture marketplace for farmers"* leads with the FAO.

3.5.3 Non-blocking by construction

The collector runs on a background thread started **before** generation begins, so by the time the model finishes, retrieval is usually already complete. A bounded wait follows, and every exception is swallowed. A dead search backend, an exhausted quota, or a malformed result costs the user their sources and nothing else. This is also why the search API key is optional: without it the system falls back to the curated tiers and still emits three to five links.

3.6 Design of the Model-Selection Layer

The model layer exists to keep a single call fast and reliable on a free-tier provider. It makes four decisions.

Latency-aware selection. Every call records its time-to-first-token per model, smoothed as a running mean. Traffic is then weighted by observed speed with a steep geometric decay by rank, rather than rotated uniformly. This decision was made on the basis of measurement, not intuition — see 3.7.

Slow-model quarantine. A model whose smoothed first-token time exceeds both a fixed threshold and a multiple of the fastest known model is benched for five minutes. Ranking alone was insufficient: with a small pool, even a heavily down-weighted slow model still received double-digit traffic, and one very slow request is a failed product.

Lite-model demotion. A lite-tier model is ranked behind every full model regardless of observed speed. Pure latency ranking placed roughly half of all traffic on a lite model, which is a real quality regression for reports that depend on judgement. Speed must not silently buy a quality drop; lite models remain the overflow when every full model is benched.

Quota rotation and bounded retries. A quota error benches that model for its provider-reported window and the request transparently retries on another. Retries are capped at two attempts, two seconds apart, and fire only for genuine transient failures. Errors indicating an authentication failure, permission denial, unknown model or malformed request are classified as permanent and surfaced immediately, since no number of retries can resolve them.

3.7 Measurement-Driven Optimisation

The optimisation pass began by instrumenting the request path to time each phase separately — request received, model call started, first token, final token, sources attached, response complete — rather than guessing. The benchmark harness is retained in the repository and is described in Chapter Four.

The measurement contradicted the initial hypothesis. Prompt sizes were small (about 2.3k characters), source retrieval cost approximately zero, and no retry cycles were firing. The actual cause was model selection: the pool rotated blindly across sibling models that the same provider served at dramatically different speeds. Measured on a short prompt, first-token latency ranged from 0.8 seconds to 28 seconds, and roughly a third of all requests were routed to the slowest model. This is what produced a 2-to-95-second spread in time-to-first-token across identical requests.

Two further causes were identified and addressed. Output volume dominated generation time, fixed by the per-feature ceilings and prompts that state explicit word budgets. Pre-answer reasoning on the model contributed several seconds of time-to-first-token, reduced by setting the thinking level to low.

The result is set out in Chapter Four. The methodological point is the one worth carrying forward: the assumed bottleneck was prompt design, and the measured bottleneck was model routing. Only measurement distinguished them.

3.8 Report Generation

Venture Intelligence follows a fixed nine-section structure: Executive Summary (verdict and reasoning) · Opportunity · Target Customers · Market (one paragraph plus a three-row TAM/SAM/SOM table) · Competition (incumbents, pricing, and the gap a new entrant could exploit) · Business Model · Key Risks (each with its mitigation) · Validation Plan (tests ordered cheapest-first, each naming what result would falsify the idea) · Recommended Next Steps · Sources.

Company X-Ray follows eight sections: Company Overview · Product and Target Market · Business Model · Competitive Position · Strengths · Weaknesses and Risks · Opportunities · Key Takeaways · Sources, with **sentiment** and **metrics** variants restructuring the report around perception and KPIs.

Discovery emits concise opportunity cards rather than a report, each framed explicitly as a hypothesis worth validating rather than a finding.

All three terminate with the same Sources section, containing three to five real, deduplicated, clickable links.

3.9 System Interface

The interface was designed around a minimal, chat-style experience:

- **Landing (hero):** a pinned platform banner; a headline that changes with the mode; a segmented toggle between **Venture Intelligence** (default) and **Company X-Ray**; one input whose placeholder adapts to the mode.
- **Workspace:** chat-like view — user request bubble, then the streamed response rendered as Markdown with tables and clickable source links. A compact status indicator shows while the first tokens arrive; progressive rendering is the normal case, not a special one.
- **Sidebar:** collapsible, with new chat, library, explore, discover, and settings, plus a searchable chat history grouped by recency with a Pinned section and per-item pin/delete actions.
- **Discover:** a focus input and a scan action producing opportunity cards; each card carries a one-click **validate** action that launches the idea directly into Venture Intelligence. Scans persist per account.
- **Library:** a gallery of completed reports with previews.
- **Settings:** live system status from `/api/health`, a manual scheduler trigger, theme switching (dark/light/system), and history management.
- **Authentication:** an account screen with email/password, Google sign-in, and a two-step password reset. On laptop and desktop widths the sign-in and register views are laid out as two columns with a product hero, the hero and the form swapping sides between the two modes; below 1280px the hero is not rendered and the form occupies a single centred column.
- **Post-authentication routing:** entering a signed-in state — by signing in, or by loading the app with an existing session — always lands on a new chat. History is retained and listed in the sidebar; only the selection is cleared.

- **Persistence & theming:** history persists in `localStorage` and syncs to the server; light and dark palettes are both supported.

3.10 Technology Stack

Only technologies actually used by the implementation are listed.

Layer	Technology	Purpose
Programming Language	Python (v3.13)	Backend development and AI integration
Web Framework	FastAPI	REST API with SSE streaming
ASGI Server	Uvicorn (standard)	Application server and SSE transport
Frontend Library	React 19	User interface
Language	TypeScript (~6.0)	Frontend type safety
Build Tool	Vite (v8) + @vitejs/plugin-react	Dev server and production bundling
CSS	Tailwind CSS v4	Utility-first styling
UI Primitives	shadcn/ui on Radix UI	Button, dialog, select, sheet, tooltip and related primitives
Utilities	class-variance-authority, clsx, tailwind-merge	Variant handling and class composition
Markdown	react-markdown + remark-gfm	Report rendering with tables and links
Icons	lucide-react	Interface iconography
Theming	next-themes	Dark/light/system colour scheme
Notifications	sonner	Toast notifications
Fonts	@fontsource-variable (Geist, JetBrains Mono, Newsreader)	Self-hosted variable fonts
Linting	oxlint	Frontend linting
AI / ML	Google Gemini via google-genai (default gemini-3.8-flash)	Reasoning and report generation
Agent Framework	Agno (>=3)	Agent construction and streaming
Source Retrieval	Firecrawl (optional)	Live web search for source links
HTTP Client	httpx	Google OAuth token and profile exchange
Authentication	bcrypt + PyJWT	Password hashing and session tokens
Persistence	SQLite via libsql-experimental, or Turso	Chats, exchanges, watchlist, accounts
Email	Python smtplib (stdlib)	Password-reset delivery
Streaming	Server-Sent Events	Real-time delivery of report deltas
Environment	python-dotenv	API key and configuration loading

Layer	Technology	Purpose
Version Control	Git	Project management and versioning

3.11 Testing Approach

The system was tested at several levels:

- **Import and syntax verification** of backend modules, with `pyflakes` for unused imports and undefined names.
- **Launch testing**: backend health endpoint and frontend production build.
- **Streaming verification**: live SSE runs confirming ordered `status`, `stage_start`, `delta` and `done` events.
- **Workflow testing**: Venture Intelligence against a real business idea; Company X-Ray against a real company; Discovery against a real sector focus.
- **Source-integrity testing**: automated checks that every report's Sources section contains between three and five deduplicated, real URLs, across a range of query types, under three conditions — no search backend, a search backend returning a single result, and a search backend returning only duplicates. Relevance is asserted directly, so that a fintech query is led by a fintech regulator, an agricultural query by the FAO, and a healthcare query by the WHO.
- **Degradation testing**: simulated source-retrieval failure confirms the report still streams and no Sources block is fabricated.
- **Error handling**: verification that a failed generation surfaces as a clear SSE `error` event rather than a silently blank report, that a run producing no output raises rather than closing a successful-looking blank, and that missing API keys produce actionable messages.
- **Authentication testing**: password-reset tokens verified single-use, one-hour expiry, one live token per user, hashed-at-rest, short-password rejection, and an identical response for known and unknown addresses.
- **Regression suites**: offline scripts run against stubbed agents, so they consume no provider quota. They cover source integrity, truncation detection, retry classification, model-selection weighting, and discovery block parsing. A separate live probe exercises a real venture run end to end.
- **Latency benchmarking**: a retained harness times each phase of a real request separately, so optimisation targets the measured bottleneck.

3.12 Summary

This chapter described the methodology: the single-analyst architecture and the reasoning behind it, the shared honesty rules, the three-tier source-collection design and its source-integrity guarantee, the latency-aware model-selection strategy, the measurement process that identified the true bottleneck, the report formats, the interface design, the technology stack, and the testing approach. The next chapter presents the implementation and the measured results.

CHAPTER FOUR

SYSTEM IMPLEMENTATION AND RESULTS

4.1 Introduction

This chapter describes the implementation of the Thrace platform and presents the results obtained. It covers the project structure, the shared intelligence layer, the model layer, the three feature prompts, the backend API with SSE streaming, the authentication flow, the user interface, the reports the system generates, and the measured latency. Testing results are tabulated at the end of the chapter.

The implementation reflects the design position stated in Chapter One:

> **Fast intelligence first.**

Concretely, this means one analyst per request with no tools on the request path, a per-feature output ceiling, and source collection that runs on a background thread and is architecturally incapable of delaying the report.

4.2 Project Structure

```
startup-intelligence-platform/
■■■■ backend/
■ ■■■■ intelligence.py # 621 lines. Shared layer: analyst rules,
■ ■ # stream_agent, error classification,
■ ■ # truncation guard, SourceCollector
■ ■■■■ agents.py # 1353 lines. Model layer + prompts:
■ ■ # latency-aware selection, quota
■ ■ # rotation, quarantine; feature prompts;
■ ■ # monitoring, digest and Q&A agents
■ ■■■■ main.py # 573 lines. FastAPI server, SSE endpoints,
■ ■ # auth routes, sync, watchlist, discover
■ ■■■■ auth.py # 325 lines. bcrypt, JWT, Google OAuth,
■ ■ # password reset
■ ■■■■ store.py # 569 lines. SQLite (local or Turso)
■ ■■■■ scheduler.py # 135 lines. Background re-validation + digest
■ ■■■■ requirements.txt
■■■■ frontend/
■ ■■■■ public/thracehero.webp # Auth-screen hero image
■ ■■■■ src/
■ ■■■■ App.tsx # Shell, routing, view state
■ ■■■■ components/
■ ■ ■■■■ landing.tsx # Hero: mode toggle + adaptive input
■ ■ ■■■■ workspace.tsx # Chat-like streaming report view
■ ■ ■■■■ discover.tsx # Opportunity scan + idea cards
■ ■ ■■■■ sidebar.tsx # Collapsible nav + grouped history
■ ■ ■■■■ library.tsx # Saved reports gallery
■ ■ ■■■■ explore.tsx # One-click example prompts
■ ■ ■■■■ settings.tsx # Status, scheduler trigger, theme, data
■ ■ ■■■■ auth.tsx # Sign-in / register / password reset
■ ■ ■■■■ status-steps.tsx # Generation status indicator
■ ■ ■■■■ pipeline-steps.tsx # Stage tracker component
■ ■ ■■■■ markdown.tsx # Report rendering (tables, links)
■ ■ ■■■■ logo.tsx
■ ■ ■■■■ ui/ # 16 shadcn/Radix primitives
■ ■■■■ lib/
■ ■■■■ api.ts # SSE reader + typed clients
```

```

■ ■■■ use-analyze.ts # Run state machine, persistence, polling
■ ■■■ discover-store.ts # Discovery scan state
■ ■■■ auth.tsx # Session context
■ ■■■ utils.ts # Class-name helper
■■■ scripts/
■ ■■■ bench_intelligence.py # Per-phase latency harness
■ ■■■ test_xray_guard.py # Regression suite
■ ■■■ test_venture.py # Live end-to-end probe
■ ■■■ test_discovery_ideas.py
■ ■■■ test_quota_failfast.py
■ ■■■ render_report.py # This report → .docx / .pdf
■■■ start.sh / stop.sh
■■■ .env # API keys and configuration
■■■ report/ # This project report

```

4.3 Implementation of the Shared Intelligence Layer

`backend/intelligence.py` is the strategic core of the platform and is shared verbatim by all three features.

Analyst rules. A single system prompt, passed as the agent description, constrains output for every feature. Its load-bearing clause is the second:

```

- Never fabricate sources, URLs or citations. Another process attaches
real links to the report for you; your job is the analysis, not the
sourcing. Do not print a "Sources" section and do not mention that
sources were or were not checked.

```

Telling the model that links are supplied for it removes any incentive to produce one.

Streaming. `stream_agent` iterates the agent's event stream and yields `delta` events. On a transient failure it retries once on a fresh agent, first emitting a `reset` event so the client discards partial text rather than showing two stitched attempts.

Error classification. `is_permanent_error` matches authentication failures, permission errors, unknown model ids and malformed requests; `is_transient_error` matches capacity and quota errors. An `_error_text` helper normalises error content, which the framework sometimes populates with a non-string object that would otherwise be rendered to the user as a raw HTTP response object.

Truncation guard. `looks_truncated` inspects the final line of a report and reports whether generation appears to have been cut mid-sentence. It is biased towards silence: headings, table rows, blockquotes and fences legitimately end without punctuation, and a short bolded label is a deliberate fragment. Only a long plain-prose line ending on a bare word is treated as truncation. A report that trips the guard is logged with its length and the applicable ceiling.

4.4 Implementation of Source Collection

`SourceCollector` runs on a daemon thread. Its lifecycle is deliberately simple: `start()` is called **before** generation begins, and `results()` is called after the report is complete, with a bounded wait.

`_run` populates the list in three tiers, deduplicating by URL and never exceeding the configured maximum. `_official_fallback` scores each of the 21 curated official bodies against the query, weighting sector terms above catch-alls and boosting a local regulator when the query names its country. If the result is still short, broad cross-sector bodies fill it to the floor.

`render_sources` re-applies the same rules at the boundary, so a caller that bypassed the collector still cannot emit a stub or an over-long list. An empty list renders nothing rather than a fabricated block.

4.5 Implementation of the Model Layer

`agents.py` owns the model client. Four decisions are implemented.

Latency tracking. `_record_latency` folds each call's time-to-first-token into a running mean per model, and quarantines a model that is both slower than a fixed threshold and a multiple of the fastest known model.

Weighted selection. `_next_model` sorts usable models by an effective latency — with the lite-model penalty applied — and walks a cumulative weight distribution with a 0.15 decay per rank. The fastest usable model therefore receives the large majority of traffic and the tail a small fraction, so a slow model no longer costs the user a very long wait on a predictable fraction of requests.

Lite-model demotion. `_rank_key` adds a large penalty to any model id containing `lite`, so it sorts behind every full model regardless of measured speed.

Bounded, classified retries. Two attempts, two seconds apart, and only for errors classified as transient. Permanent errors short-circuit the retry entirely.

Output ceilings are applied per feature at model construction, and the Gemini thinking level is set to low, which removes several seconds of pre-answer reasoning from time-to-first-token.

4.6 Implementation of the Feature Prompts

Each feature supplies a distinct user prompt and an output ceiling; the system prompt and honesty rules are shared.

Venture Intelligence fixes a nine-section structure and gives each section a word budget, with an explicit instruction that no section may repeat another's content and that the sections are the whole report with no preamble or closing summary.

Company X-Ray fixes eight sections for `competitor` analyses, with `sentiment` and `metrics` variants restructuring the report around perception and KPIs. A user-supplied URL is detected and passed to the model as a hint about which company is meant, with an explicit instruction that the site was not visited and nothing only visible on it may be claimed.

Discovery proposes exactly four opportunities and enforces the framing rule the product depends on: ideas are **hypotheses worth validating**, never findings. The prompt directs the analyst to say "appears worth validating" and "the first thing to test", never "the market is proven" or "demand is confirmed", and to state the specific thing that should be tested first. It runs with no tools, so it cannot be delayed by a search backend.

4.7 Implementation of the Backend API

The FastAPI backend exposes 24 endpoints across intelligence, authentication and persistence.

- GET `/api/health` — readiness, active model, feature list, analyst roster. A model key is required; the search API key is explicitly **not**, because source retrieval is optional by design.
- POST `/api/venture` — { "idea": string } → status, stage_start, delta events, then done or error.
- POST `/api/analyze` — { "company", "analysis_type" } → the same event shape.
- POST `/api/discover` — { "focus"? } → delta and idea events, then done.
- POST `/api/ask` — { "content", "question" } → an answer grounded strictly in the supplied report.

All streaming endpoints execute work on a background thread feeding a queue, so the server remains responsive while events are flushed. Idle periods emit SSE comment lines to keep intermediaries from closing the connection. Validation returns clear HTTP 400/503 errors.

4.8 Implementation of Authentication

Accounts support three paths. **Email and password** registration and sign-in use bcrypt hashing and JWT sessions with a two-week expiry; a bearer token is sent by the client and resolved by a FastAPI dependency. **Google OAuth 2.0** runs in a popup with a single-slot anti-forgery state, posting the session token back to the app via `postMessage`. **Password reset** uses a single-use, time-limited token: only its SHA-256 digest is stored, so the database never holds a usable reset link; the forgot-password endpoint returns an identical response for known and unknown addresses, so it cannot be used to discover which emails have accounts; requests are throttled per address; and delivery goes over SMTP when configured.

4.9 Implementation of Persistence and the Scheduler

`store.py` abstracts SQLite behind a narrow interface and works unchanged against a hosted Turso database, normalising the differences between the local and remote drivers behind the same connection wrapper. Seven tables are maintained: users, runs, exchanges, watchlist, meta,

password resets and discovery scans.

`scheduler.py` runs a background thread inside the FastAPI process. It re-validates watched chats when due using the monitoring agent, appending update exchanges server-side, and generates the in-app intelligence digest when new monitoring updates exist. A manual trigger is exposed for demonstration.

4.10 Implementation of the User Interface

The React interface implements the design described in Chapter Three. The interface displays a compact status indicator while the first tokens arrive, after which the streamed report is the primary presentation.

Workspace staging. A stage-tracker component is retained in the codebase, but the current request path emits a single report stage and never signals the five research stages the component can display. The workspace therefore gates the tracker on whether any stage has actually activated, and falls back to the status indicator — so the tracker is not rendered during a normal run.

Auth screen. On widths of 1280px and above the sign-in and register views are two-column, with a product hero image on one side and the form on the other; toggling between sign-in and register swaps which side the hero occupies. Below 1280px the hero is not rendered and the form centres as a single narrow column. The hero is served as WebP.

Post-authentication routing. Signing in, and loading the app with an existing session, both clear the active chat selection and show the new-chat view. History is retained in the sidebar; only the selection is cleared, so nothing is lost.

4.11 System Output and Results

4.11.1 Venture Intelligence Report

A run on `"AI marketplace for Nigerian farmers"` produces:

```
# Venture Intelligence – AI marketplace for Nigerian farmers

## Executive Summary
Verdict: PIVOT
- Marketplace models face significant friction in Nigerian agriculture...
- Shift focus to an AI-enabled B2B supply chain optimisation tool...
- The primary value proposition is for the corporate buyer, not the farmer...

## Opportunity
## Target Customers
## Market (paragraph + TAM/SAM/SOM table, figures
labelled as estimates with assumptions)
## Competition (incumbents, pricing, the gap to exploit)
## Business Model
## Key Risks (each with its mitigation)
## Validation Plan (cheapest-first, with falsification criteria)
## Recommended Next Steps
## Sources
- [NITDA (Nigerian IT Development Agency)](https://www.nitda.gov.ng)
```

- [FAO (UN Food & Agriculture)](<https://www.fao.org>)
- [World Bank](<https://www.worldbank.org>)
- [African Development Bank](<https://www.afdb.org>)

4.11.2 Company X-Ray Output

Eight sections ending in the same Sources block:

```
# Paystack – Company X-Ray
## Company Overview
## Product & Target Market
## Business Model
## Competitive Position
## Strengths
## Weaknesses & Risks
## Opportunities
## Key Takeaways
## Sources
```

4.11.3 Discovery Output

Four opportunity cards, each with a title, a one-line idea runnable as a venture prompt, and a rationale stating the problem, the target customer, a possible business model, and the first thing to test. A real run on `"business opportunities in Nigeria for young people"` returned ideas including solar-powered cold-chain storage for peri-urban markets, a shared hardware and training centre for creative freelancers, apprenticeship matching for informal trades, and on-demand spare-parts delivery to mechanics. The specificity of the local references indicates the model is reasoning from pretrained knowledge rather than retrieving, consistent with the no-tools design.

4.11.4 Measured Latency

Recorded with `scripts/bench_intelligence.py`, which times request receipt, model call start, first token, final token, source attachment and completion separately. Figures are from the Gemini free tier and vary with model load.

Feature	Time to first token	Total	Output	Sources
Venture Intelligence	2–8s	8–14s	~5.5k chars	3–5
Company X-Ray	2–8s	6–10s	~4.5k chars	3–5
Discovery	2–6s	5–10s	~3.4k chars	3–5

The reduction is attributable to three changes, in order of measured impact: latency-aware model selection, which removed the routing of a predictable fraction of requests to a model served at around 28s to first token; per-feature output ceilings and prompts that state explicit word budgets, which cut Venture output substantially; and a reduced thinking level, which removed several seconds of pre-answer latency.

> **Measurement caveat.** The Gemini free tier permits 20 requests per day per > model. Sustained benchmarking exhausts this allowance, and failures observed > during measurement are that quota rather than application faults. The figures > above come from runs that

completed.

4.11.5 Testing Results

Test	Method	Result
Syntax and import	pyflakes + module import	Passed — no unused or undefined names
Backend launch	Uvicorn start + <code>/api/health</code>	Passed — model, features and analyst roster reported
Frontend build	<code>tsc -b && vite build</code>	Passed (typecheck + bundle)
Venture streaming	<code>POST /api/venture</code> over SSE	Passed — <code>status/stage_start</code> , then <code>deltas</code> , then <code>done</code>
X-Ray streaming	<code>POST /api/analyze</code> over SSE	Passed — ordered events
Discovery streaming	<code>POST /api/discover</code> over SSE	Passed — 4 <code>idea</code> events emitted progressively
Source count and integrity	14 query types × 3 backend conditions	Passed — 3–5 deduplicated real links in every case
Source relevance	Asserted per sector	Passed — fintech→CBN, agriculture→FAO, health→WHO
Search-backend outage	Simulated retrieval failure	Passed — report streamed, no fabricated Sources block
Empty report handling	Stubbed silent agent	Passed — raises rather than closing a blank report
Retry classification	Auth, bad-request, capacity and quota inputs	Passed — permanent errors not retried, transient ones are
Model selection	Simulated pool with known latencies	Passed — slow model receives a minimal share; quarantined model receives none
Truncation detection	Markdown/prose cases	Passed — detects cut prose, ignores structural endings
Password reset	Live end-to-end	Passed — single use, expiry enforced, no user enumeration, old password rejected
Regression suites	3 offline scripts	Passed — no provider quota consumed
Live probe	<code>test_venture.py</code> against a running server	Passed — 54 delta events, report generated, no errors
Input validation	Empty and oversized idea	Passed — HTTP 400 with clear detail

4.12 Summary

This chapter described the implementation: the shared intelligence layer and its honesty mechanisms; the three-tier source collector; the latency-aware model layer; the three feature prompts; the SSE API and its supporting endpoints; the authentication and persistence layers;

and the interface. Measured results confirm streaming delivery in seconds rather than minutes, concise output, and 3–5 genuine sources on every report. The next chapter discusses these results.

CHAPTER FIVE

DISCUSSION

5.1 Introduction

This chapter discusses the findings of the study. It examines the extent to which the project achieved its aim and objectives, compares Thrace with the related systems identified in Chapter Two, discusses strengths and limitations, and reflects on the challenges encountered during implementation and how they were resolved.

The discussion is framed by the design position the project settled on:

> **Fast intelligence first.**

Much of what follows — the response times, the source-integrity guarantee, the honest framing of Discovery, and the measurement-driven optimisation — is a consequence of that one decision rather than an independent addition to it.

5.2 Achievement of Aim and Objectives

The aim was to design and implement a platform that validates business ideas and generates decision-ready venture planning intelligence quickly enough to be used conversationally — returning a structured report in seconds rather than minutes, with a small number of real source links, and without fabricating evidence. The completed system achieves this aim.

Objective 1: Design a single fast intelligence architecture serving three capabilities.

Achieved. Venture Intelligence, Company X-Ray and Discovery run through one shared layer with a common system prompt, common streaming behaviour and common source collection, differing only in their user prompt and output ceiling. Adding a fourth feature would require a new prompt and a ceiling, not a new pipeline.

Objective 2: Develop the Venture Intelligence report structure. Achieved. The report covers an executive verdict, the opportunity, target customers, market sizing with stated assumptions, the competitive landscape and the gap a new entrant could exploit, the business model and its deciding metric, key risks with mitigations, a validation plan ordered cheapest-first with falsification criteria, and recommended next steps.

Objective 3: Implement lightweight source collection with an integrity guarantee.

Achieved, and the guarantee is structural rather than procedural. Three independent mechanisms must all fail for a fabricated URL to reach a user: the model is forbidden by system prompt from writing one; the renderer is the only component that emits a link and accepts only structured source objects; and every URL is either a real search result or a known official organisation's root domain held in source code. Source retrieval never blocks generation, and every failure path is swallowed.

Objective 4: Generate structured reports streamed progressively. Achieved. Every feature streams Server-Sent Events, and the first tokens reach the interface in 2–8 seconds. The report renders as it is written.

Objective 5: Build an interactive, modern user interface. Achieved. The React interface delivers a minimal chat-style experience with an adaptive Venture/X-Ray toggle, a collapsible sidebar with searchable and pinnable history, a report library, an opportunity discovery view, explore prompts, settings, account-based sync with password recovery, and light and dark theming.

Objective 6: Measure latency by phase and optimise the measured bottleneck. Achieved, and this objective produced the most consequential result in the project. The measurement contradicted the working hypothesis: prompt design was assumed to be the cause of the latency and was not. See 5.6.

5.3 Comparison with Related Systems

Two comparisons matter. The first is with the **multi-agent intelligence demonstrators** reviewed in Chapter Two, whose architecture directly informed the Company X-Ray capability. The second is with **general-purpose conversational assistants**, which are the closest functional competitor.

The comparison is limited to what the literature review records about these systems. Where the review is general — as it is for the demonstrators, which are treated as a class — the comparison is correspondingly general, and no specific unnamed system is characterised in detail.

5.3.1 Comparison with multi-agent company-analysis demonstrators

Dimension	Multi-agent demonstrators	Thrace
Purpose	Produce competitor, sentiment and metrics reports for a given company	Validate a business idea, analyse a company, and propose opportunities worth validating
Target users	Analysts and investors researching existing companies	Founders at the idea stage, plus analysts and investors
Core capabilities	Company analysis across three dimensions	Idea validation, market and competitor analysis, business modelling, risk, validation planning, company analysis, opportunity discovery
Architecture	Teams of tool-equipped agents, often with a coordinating team and a synthesis pass	One analyst per request, no tools, shared across all features
Level of autonomy	Autonomous execution of a research engagement	Autonomous generation of an analysis from pretrained knowledge; optional background source retrieval
Venture validation	Not addressed	Primary capability

Dimension	Multi-agent demonstrators	Thrace
Company analysis	Primary capability	Retained as a secondary capability
Opportunity discovery	Not addressed	Present, with ideas framed as hypotheses worth validating
Report generation	Dimension-specific reports	One fixed structure per feature, consistent across runs
Source handling	Agents retrieve and cite during the run	Model forbidden from writing URLs; 3–5 real links attached by the backend, never blocking generation
User workflow	Submit a company, wait for a research engagement	Submit and read progressively; results in seconds; follow-up questions answered from the report
Response time	A research engagement; minutes	Seconds

The decisive difference is not output quality but **subject and timescale**. The demonstrators analyse companies that already exist; the venture question — whether an idea should be pursued at all — is both earlier in the lifecycle and the one that benefits most from a fast turnaround, because a founder's actual workflow is comparing several ideas in one sitting. A research engagement measured in minutes forecloses that workflow.

5.3.2 Comparison with general-purpose conversational assistants

Dimension	Conversational assistants	Thrace
Purpose	General question answering	Venture planning and opportunity discovery, structured as a product
Target users	General public	Founders, and specifically those outside the coverage of market-research tools
Core capabilities	Broad, prompt-determined	Fixed analytical frame: opportunity, customers, market, competition, model, risk, validation, next steps
Architecture	General conversational model	Purpose-built FastAPI + streaming frontend with a constrained analyst
Level of autonomy	Responds when asked	Generates a complete structured report from one prompt; watchlist re-validation and digests run on a schedule
Venture validation	Possible if prompted for it	Default behaviour, consistently structured
Company analysis	Possible if prompted for it	A first-class mode with a dedicated structure
Opportunity discovery	Possible if prompted for it	A dedicated view producing four cards, each launchable into validation
Report generation	Format varies with the prompt; two runs are not comparable	Fixed structure; runs are comparable

Dimension	Conversational assistants	Thrace
Source handling	May produce plausible but unverified URLs; no boundary between knowledge and verification	Model cannot write URLs; 3–5 verified real links attached by the backend
User workflow	The user must engineer the prompt to obtain structure, depth and constraints	One prompt; follow-up questions answered strictly from the report
Continuity	Analyses not retained or comparable over time	Chats persist locally and server-side; watchlist re-validation and periodic digests

The differences are structural rather than model-capability differences, and this is the honest framing. A capable user of a general assistant can obtain a comparable analysis. What the assistant does not provide is a consistent analytical frame across runs, a boundary between what the model knows and what has been verified, a retained and watchable history, or the discovery capability for a user who does not yet have an idea.

5.3.3 Comparison with other related tools

Compared with **market-research databases** (CB Insights, Crunchbase), Thrace inverts the direction of analysis: rather than querying a fixed database about existing, mostly funded, developed-market companies, it reasons about the user's own idea, including informal local competition that no database catalogues.

Compared with **business-planning software**, Thrace produces an analytical view rather than a template to be filled in, and the Validation Plan section replaces the founder's assumptions with named falsification tests.

Compared with **social-listening tools** such as Brandwatch, Thrace addresses a different actor — an aspiring founder rather than an established brand tracking its own mentions — and covers multiple analytical dimensions rather than one.

5.4 Strengths of the System

- **Response time appropriate to the task.** A complete, structured report in 8–14 seconds changes what a founder can do with the tool: several ideas can be compared in one sitting, which is the workflow the product is for.
- **Source integrity as a structural guarantee.** Not a prompt instruction but an architectural property, verified by automated test under three backend conditions.
- **Graceful degradation.** Source retrieval, individual models in the pool, and the entire search backend can each fail without taking the report down. Source collection, in particular, is architecturally incapable of blocking the response.
- **Measured optimisation.** The largest performance win came from instrumenting the system and trusting the measurement over the hypothesis.

- **Consistent structure.** A fixed section list means reports are comparable across runs and across users.
- **Honest framing in Discovery.** Opportunity proposals are explicitly hypotheses, which prevents the most damaging failure mode of generative business advice.
- **Localisation.** Reasoning about specific cities, local currencies, informal competitors and national regulatory requirements, with a source layer that surfaces the relevant local regulator.
- **Product-grade interface.** Streaming, searchable and pinnable history, a report library, discovery, theming, accounts with password recovery, and server-side sync.

5.5 Limitations of the System

- **No live evidence on the normal path.** This is the central trade-off and it should be stated plainly. The analyst reasons from pretrained knowledge, so a market figure or competitive claim may be out of date. The design mitigates this — figures must be labelled as estimates, judgement must be hedged, and sources are attached for orientation — but it does not eliminate it. A user needing current figures must verify them.
- **Curated-source ceiling.** While no search provider has credits, sources come from a curated list of official bodies. Relevance is scored and localised, but the list cannot reflect a company-specific or event-specific source.
- **LLM variability.** Analyses vary between runs. A fixed section structure limits but does not remove this.
- **Output ceilings trade depth for speed.** The validation and planning sections are necessarily shallower than a multi-page feasibility study. Follow-up questions partially recover this.
- **Free-tier quota ceiling.** The Gemini free tier permits 20 requests per day per model, which limits sustained use. This is a provider constraint, not an architectural one, and a paid key removes it.
- **No analytical history.** Chats are retained, but there is no model of how an assessment changed over time, which is what a founder monitoring a venture would want.
- **Sessions survive a password reset.** JWTs are stateless, so outstanding sessions are not revoked when a password is reset. Closing this requires a token-version column checked on session resolution.
- **Discovery quality is unvalidated.** The feature proposes opportunities but

provides no feedback mechanism, so the system cannot learn which of its proposals were useful.

- **A vestigial stage tracker.** A stage-tracker component remains in the codebase from a multi-stage report format, but the current request path never signals those stages, so it is gated off and never rendered. It is dead weight that should be removed.

5.6 Challenges Encountered

- **Silent agent failures.** The agent framework returns failed runs as objects containing the error text rather than raising exceptions. This initially caused output to appear successful while containing error messages. It was diagnosed during live testing and fixed with an explicit status-and-content check, and the check was retained as a regression test.

- **A blank report presented as success.** A run that completed without emitting a single token closed the stream as though it had succeeded, leaving the user with an empty page. The fast path now raises rather than closing a successful-looking blank report, and a regression test covers it.

- **The assumed bottleneck was the wrong one.** The working hypothesis was that prompt design and output volume caused the latency. Instrumenting the request path per phase showed that prompts were small, source retrieval cost approximately zero, and no retries were firing. The actual cause was model selection: the pool rotated blindly across sibling models that the same free tier served between under a second and around half a minute to first token, sending a predictable fraction of all requests to the slowest. Only measurement distinguished the two, and the fix — latency-aware weighting plus a quarantine for pathologically slow models — was worth more than every other change combined.

- **Optimising speed by blindly trusting latency.** The first implementation of latency-aware selection sent roughly half of all traffic to a lite-tier model, because it was the fastest available. That was a real quality regression for reports that depend on judgement. Lite models are now ranked behind every full model and remain the overflow only.

- **Truncation as a side effect of the speed fix.** Lowering the output ceiling for latency caused reports to be cut off mid-sentence, which reads as a broken product rather than a concise one. The ceilings were retuned to complete, a truncation detector was added, and a regression test covers both the detection and the false-positive cases.

- **A truncation fix that was wrong.** An initial attempt to repair a truncated report issued a second call asking the model to continue the final sentence. This was abandoned on inspection: the client has no way to replace text it has already streamed, so a continuation would be appended after the dangling clause and read as garbled text. The condition is now detected and logged rather than patched.

- **Framework API change (Agno v2 → v3).** The tool configuration parameter

names changed between versions, requiring the tool wrapper to be updated.

- **Error objects leaking into the interface.** The framework sometimes places a non-string object in an error field, which surfaced to the user as a raw HTTP response object. An error-text helper now normalises the message.
- **Model-provider configuration.** Testing revealed provider-credential issues, including an expired gateway key and an out-of-credit account. The model layer was made configurable so any OpenAI-compatible provider can be substituted.
- **Recovering a deleted binary asset.** A `.gitignore` entry written without a leading slash matched the tracked asset as well as the stray working copy, so the asset was never actually version-controlled and deleting it removed it from disk as well. It was recovered from a dangling git object and the ignore pattern re-anchored. The lesson is that ignore patterns intended to scope to one directory need an explicit leading slash.
- **Theme implementation.** The initial stylesheet defined identical light and dark palettes, making theme switching ineffective; distinct palettes were authored and theme-dependent visuals were made palette-aware.

5.7 Implications of the Study

The most transferable finding is methodological. The project entered its optimisation phase with a confident, plausible and wrong hypothesis about where its latency came from, and only per-phase instrumentation revealed the real cause. The eventual fix — routing requests according to measured model performance rather than rotating blindly — was worth more than every other change combined, and it was not visible from the code. This is a concrete argument that LLM systems require the same phase-level measurement discipline that distributed systems received years earlier.

The second finding concerns the relationship between thoroughness and usability. Extensive retrieval makes a system slower without necessarily making it more useful, because the analytical value of a report comes from reasoning quality rather than from the volume of material consulted. Restricting the request path to pretrained knowledge, and treating source retrieval as a background enhancement that cannot block the response, is what converted the product from a demonstration into something usable.

The third finding concerns source integrity. The prohibition on model-generated URLs is the kind of constraint that looks like a limitation in a requirements document and turns out to be the thing that makes a citation feature safe to ship at all. Removing it would have been simpler; the resulting product would have invented URLs.

5.8 Summary

This chapter confirmed that all six objectives were achieved; compared Thrace with multi-agent company-analysis demonstrators, general-purpose conversational assistants,

market-research databases, business-planning software and social-listening tools; identified strengths (response time, structural source integrity, graceful degradation, measured optimisation, honest framing) and limitations (no live evidence on the normal path, curated-source ceiling, LLM variability, free-tier quota, sessions surviving a password reset, unvalidated Discovery, a vestigial stage tracker); and documented the implementation challenges and their resolutions, notably that the assumed performance bottleneck was not the actual one. The final chapter concludes the report and recommends future work.

CHAPTER SIX

CONCLUSION AND RECOMMENDATIONS

6.1 Introduction

This chapter concludes the project report. It summarises the study, restates its contributions, and sets out recommendations for the future development of the Thrace Startup Intelligence Platform.

6.2 Summary of the Study

This project set out to design and implement a Startup Intelligence Platform for Business Discovery and Venture Planning. It was motivated by the plight of aspiring entrepreneurs — particularly in emerging markets — who must decide whether and how to pursue a business idea without access to affordable validation and planning intelligence. The literature review established that while the evidence needed for such decisions is abundant on the open web, no existing tool delivers it quickly, with a consistent analytical structure, and with a guaranteed boundary between what a model knows and what has been verified.

Thrace was designed and implemented to fill this gap. Its design rests on a deliberate position:

> **Fast intelligence first.**

The platform provides three capabilities over one shared intelligence layer. **Venture Intelligence** returns a structured report covering an executive verdict, the opportunity, target customers, market sizing with stated assumptions, the competitive landscape and the gap a new entrant could exploit, the business model and its deciding metric, key risks with mitigations, a validation plan ordered cheapest-first with falsification criteria, and recommended next steps. **Company X-Ray** applies the same layer to an existing company. **Discovery** proposes business opportunities framed explicitly as hypotheses worth validating, each launchable into validation with one click. Every report ends with three to five real, deduplicated, clickable source links.

The architecturally decisive choice is that the analyst runs with no tools at all. It cannot search or browse, so report generation is a function of the model alone and of output length alone — which makes latency predictable and keeps the failure surface small. Real source links are attached separately by the backend on a background thread that is started before generation and cannot delay it.

The source-integrity guarantee is structural. The model is forbidden by system prompt from writing a URL; a dedicated renderer is the only component that emits a link and accepts only structured source objects; and every URL is either a real search result or a known official organisation's root domain held in source code.

The system was implemented as a full-stack application — a React 19 + TypeScript + Tailwind CSS v4 + shadcn/ui frontend with a chat-style interface, collapsible sidebar, searchable and pinnable history, report library, discovery view, explore prompts, accounts with Google sign-in

and password recovery, server-side sync and theming; and a Python FastAPI backend streaming Server-Sent Events, built on the Agno framework and Google Gemini, with a latency-aware model pool, a background scheduler for watchlist re-validation and digests, and SQLite persistence. Measured results confirm Venture Intelligence at 8–14 seconds, Company X-Ray at 6–10 seconds and Discovery at 5–10 seconds, with time-to-first-token between 2 and 8 seconds.

6.3 Contributions of the Study

1. **A working, fast venture-intelligence product** demonstrating that consultant-grade validation and planning analysis can be delivered conversationally — in seconds rather than minutes — from a single plain-language prompt, and used to compare several ideas in one sitting.
1. **A source-integrity guarantee by construction.** Rather than instructing a model not to invent citations, the architecture removes its ability to: the model cannot write a URL, a renderer is the sole emitter of links, and every link traces to a search result or a curated official domain. This is reusable as a pattern for any generative system that must cite sources.
1. **A demonstrated case for measurement-driven optimisation.** The project's largest performance gain came from instrumenting per-phase latency and optimising the measured bottleneck rather than the assumed one. The assumed cause — prompt design — was not the cause; model routing was. The finding, and the quarantine and weighting strategies derived from it, transfer to any system drawing on a heterogeneous pool of models.
1. **An honest framing discipline for generative business advice.** Discovery is constrained to propose hypotheses worth validating, never findings, with explicit rules against fabricated statistics and claims of proven demand. This addresses what is arguably the most damaging failure mode of generative business advice.
1. **Graceful degradation as a design property.** Source retrieval, individual models, and the entire search backend can each fail without taking down the product, and this is achieved structurally rather than by catching exceptions at the top level.
1. **A complete product surface around the intelligence** — accounts, Google sign-in, password recovery with single-use tokens, server-side chat persistence, a watchlist with scheduled re-validation, and an in-app digest — demonstrating that the value is in the whole workflow, not only the generation step.
1. **An open, extensible codebase** with a provider-agnostic model layer and a feature-per-prompt architecture, in which a new capability requires a new prompt and an output ceiling rather than a new pipeline.

6.4 Future Enhancements

None of the capabilities in this section is implemented. They are the roadmap for making Thrace more autonomous and a stronger product. They are grouped by theme, and each is stated as a proposed direction rather than a committed feature.

6.4.1 Continuous and scheduled intelligence

- **Continuous market monitoring.** Track a venture's market, competitors and demand signals continuously rather than validating at a point in time. The existing watchlist scheduler is the natural foundation; this extends it from fixed-interval re-validation to signal-driven monitoring.
- **Automatic trend detection.** Identify emerging sectors and shifting consumer behaviour without a user prompt, and surface what changed.
- **Scheduled intelligence updates.** Replace the fixed weekly re-validation with user-defined cadences per idea, company or watchlist entry, appropriate to how fast that market moves.
- **Automated competitor monitoring.** Detect new entrants, pricing moves, funding events and product launches for tracked companies, and report the delta rather than regenerating a full analysis.
- **Notification and alerting.** Push or email alerts when a monitored signal crosses a user-defined threshold, so the value arrives without the user opening the application.

6.4.2 Memory and learning

- **Persistent startup and idea memory.** Retain the full analytical history of an idea and track how its assessment evolved, so the platform can report what changed and why rather than only what is currently true.
- **Learning from user feedback.** Capture which recommendations were acted on and which dismissed, and use that signal to calibrate Discovery and to weight the emphasis of future analyses.
- **Personalised recommendations.** Tailor Discovery and analysis depth to a user's sector, geography, capital constraints and stage.

6.4.3 Opportunity intelligence

- **Opportunity scoring.** Apply a transparent, weighted score to each discovered opportunity so that four cards can be ranked rather than merely read, and so that scores are comparable between runs and users. The decomposition principle from Section 2.2.5 applies, with the caution recorded there about presenting a precise number built on estimated inputs.
- **Validation-experiment design.** Generate cheap, concrete falsification

tests automatically, including sample sizes, success thresholds and an estimated cost per test, extending the current Validation Plan section.

- **Autonomous follow-up research.** Trigger deeper retrieval automatically where the fast pass found the evidence too thin — the selective middle ground between the current "never research" position and unconditional retrieval.

6.4.4 Trust and verification

- **Stronger source verification.** Confirm that each cited URL resolves and supports the claim attributed to it, and score source quality so that a regulator's publication outranks an unattributed blog post.
- **Claim-level provenance.** Attach evidence to individual claims rather than to a report as a whole, so a reader can see exactly which statement rests on which source.
- **Recency weighting.** Prefer recent sources for fast-moving markets, and date-stamp the report so a reader can judge how current it is.

6.4.5 Product surface

- **Report export** to PDF, DOCX and CSV, for submission to banks, investors and grant programmes.
- **Session revocation** on password reset, closing the stateless-JWT gap identified in Section 5.5.
- **Team and shared workspaces** — collaborative reports, shared watchlists and shared opportunity collections.
- **Public API** allowing third-party tools to request a report programmatically.
- **Mobile-optimised Discovery and Library**, since the two-column authentication layout establishes a desktop-first pattern that will need a deliberate mobile treatment as the feature set grows.

6.4.6 Maintenance

- **Remove the vestigial stage tracker.** The component remains in the codebase but is unreachable in the current request path; it should be deleted rather than left gated, to avoid misleading a future maintainer into believing the system still reports research stages.
- **Reassess the curated source list** as real search results become available, and retire the fallback once it is no longer load-bearing.

6.5 Conclusion

This project set out to place evidence-based venture planning within reach of anyone with an idea and an internet connection, and to do it fast enough that the tool changes what a founder can do. By combining a constrained analyst without tools with background source collection, Thrace converts a process that once demanded expensive consultants into a conversational service that returns a structured, sourced report in seconds.

The central design commitments are worth restating, because they are what make the product trustworthy rather than merely fast. The model is not permitted to write a URL, so citations cannot be fabricated; figures must be labelled as estimates, so precision is not invented; Discovery is constrained to propose hypotheses rather than assert findings, so the platform does not manufacture confidence; and source retrieval cannot block the response, so the promise of speed is structural rather than a matter of tuning.

The most useful lesson the project offers is a methodological one. The team entered its optimisation phase with a confident and incorrect hypothesis about the source of its latency, and only per-phase measurement identified the real cause — model routing, worth more than every other change combined. That is a finding about LLM systems generally, and it is the one most likely to be reproduced by anyone building on this architecture.